

B K Birla Institute of Engineering & Technology, Pilani

45 Days Summer Training Program

Subject: Data Science with AIML

Name: Praveen Poonia

Id: 24EBKCS087

Branch: CS

College: BKBIET Pilani

Assignment 4

Question 1.

Write a program that takes a list of numbers and prints all prime numbers in the list using loops. Do not use any built-in prime-checking functions.

Program:

```
numbers = [2, 4, 5, 8, 11, 15, 17, 20, 23]
```

```
print("Prime Numbers are:")
```

```
for num in numbers:
```

```
    if num < 2:  
        continue
```

```
    prime = True
```

```
    for i in range(2, num):  
        if num % i == 0:  
            prime = False  
            break
```

```
    if prime:  
        print(num)
```

Output:

Prime Numbers are:

2
5
11
17
23

Question 2.

Write a program that generates a multiplication table (1–10) in matrix format using nested loops.

Program:

```
for i in range(1, 11):  
    for j in range(1, 11):  
        print(i * j, end=" ")  
  
    print()
```

Output:

```
1 2 3 4 5 6 7 8 9 10  
2 4 6 8 10 12 14 16 18 20  
3 6 9 12 15 18 21 24 27 30  
...  
10 20 30 40 50 60 70 80 90 100
```

Question 3.

Write a user-defined function that accepts a list of integers and returns:

- Count of even numbers
- Sum of odd numbers
- Use loops and conditional statements inside the function.

Program:

```
def analyze_numbers(numbers):
```

```
even_count = 0
odd_sum = 0

for num in numbers:

    if num % 2 == 0:
        even_count += 1

    else:
        odd_sum += num

return even_count, odd_sum
```

```
data = [10, 15, 20, 25, 30]

even_count, odd_sum = analyze_numbers(data)

print("Count of Even Numbers =", even_count)
print("Sum of Odd Numbers =", odd_sum)
```

Output:

```
Count of Even Numbers = 3
Sum of Odd Numbers = 40
```

Question 4.

Create a function that calculates simple interest.

Use default arguments for rate and time. Demonstrate calling the function using both positional and keyword arguments.

Program:

```
def simple_interest(principal, rate=5, time=2):

    si = (principal * rate * time) / 100

    return si
```

```
print("Using Positional Arguments")
print(simple_interest(10000))

print("\nUsing Keyword Arguments")
print(simple_interest(principal=10000, rate=8, time=3))
```

Output:

```
Using Positional Arguments
1000.0
```

```
Using Keyword Arguments
2400.0
```

Question 5.

Create a Student class with attributes: name, marks. Add a method to calculate grade based on marks.

Program:

```
class Student:

    def __init__(self, name, marks):
        self.name = name
        self.marks = marks

    def calculate_grade(self):

        if self.marks >= 90:
            return "A"

        elif self.marks >= 75:
            return "B"

        elif self.marks >= 60:
            return "C"

        else:
            return "D"
```

Output:

Class Created Successfully

Question 6.

Create at least two objects and display their grades.

Program:

```
class Student:
```

```
    def __init__(self, name, marks):
```

```
        self.name = name
```

```
        self.marks = marks
```

```
    def calculate_grade(self):
```

```
        if self.marks >= 90:
```

```
            return "A"
```

```
        elif self.marks >= 75:
```

```
            return "B"
```

```
        elif self.marks >= 60:
```

```
            return "C"
```

```
        else:
```

```
            return "D"
```

```
student1 = Student("Praveen", 92)
```

```
student2 = Student("Rahul", 78)
```

```
print(student1.name, "-", student1.calculate_grade())
```

```
print(student2.name, "-", student2.calculate_grade())
```

Output:

Praveen - A

Rahul - B

Question 7.

Create a BankAccount class with private attributes for balance. Provide methods to deposit and withdraw money. Ensure balance cannot be accessed directly.

Program:

```
class BankAccount:

    def __init__(self):
        self.__balance = 0

    def deposit(self, amount):

        if amount > 0:
            self.__balance += amount

    def withdraw(self, amount):

        if amount <= self.__balance:
            self.__balance -= amount

        else:
            print("Insufficient Balance")

    def show_balance(self):
        print("Current Balance =", self.__balance)

account = BankAccount()

account.deposit(5000)
account.withdraw(2000)

account.show_balance()
```

Output:

Current Balance = 3000

Question 8.

Write a program that takes two numbers from the user and performs division.

- Handle the following exceptions:
- Division by zero
- Invalid input type

Program:

try:

```
num1 = int(input("Enter First Number: "))  
num2 = int(input("Enter Second Number: "))
```

```
result = num1 / num2
```

```
print("Result =", result)
```

except ZeroDivisionError:

```
print("Division by zero is not allowed.")
```

except ValueError:

```
print("Invalid input type.")
```

Output 1:

```
Enter First Number: 20  
Enter Second Number: 5  
Result = 4.0
```

Output 2:

```
Enter First Number: 20  
Enter Second Number: 0  
Division by zero is not allowed.
```

Question 9.

Write a program that:

- Creates a file
- Writes student data (name and marks)
- Reads and displays the content

Program:

```
file = open("student.txt", "w")

file.write("Name: Praveen Poonia\n")
file.write("Marks: 95")

file.close()

file = open("student.txt", "r")

content = file.read()

print(content)

file.close()
```

Output:

```
Name: Praveen Poonia
Marks: 95
```

Question 10.

Write a program that reads numbers from a file and calculates:

- Total
- Average
- Handle the case if the file does not exist.

Program:

try:

```
file = open("numbers.txt", "r")
```

```
total = 0
```

```
count = 0
```

```
for line in file:
```

```
    total += int(line)
```

```
    count += 1
```

```
average = total / count
```

```
print("Total =", total)
```

```
print("Average =", average)
```

```
file.close()
```

except FileNotFoundError:

```
    print("File does not exist.")
```

Output:

```
Total = 150
```

```
Average = 30.0
```

Output (If File Does Not Exist):

```
File does not exist.
```